Data Structures Fall 2023: Stacks and its Applications

Task One :

Make a stack class that implements all the following functions, it should be templatized.

1) Stack (int ignored = 0)
   - Results: Constructor. Creates an empty stack.

2) ~Stack ()
   - Results: Destructor. Deallocates (frees) the memory used to store a stack.

3) void push (const DataItem)
   - Results: Push the element at the top of the stack.

4) Void pop ()
   - Results: Returns the element from the top of the stack.

5) element Peek ()
   - Results: Return the element at the top of the stack.

6) void clear ()
   - Results: Removes all the elements from a stack.

7) Bool isEmpty ()
   - Results: Returns true if a stack is empty. Otherwise, returns false.

Task Two:

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.
An input string is valid if:
Open brackets must be closed by the same type of brackets.
Open brackets must be closed in the correct order.
Every close bracket has a corresponding open bracket of the same type.

Task Three:

You are given a maze represented as an N*M binary matrix of blocks, and a rat is initially located at (0, 0) in the maze, denoted as maze[0][0]. The objective of the rat is to reach a specific location in the maze, represented by the coordinates (fx, fy), where there is food.

In the maze matrix, each cell is either a 0 (indicating a dead end) or a 1 (indicating a valid path). The rat can move in any of the four cardinal directions (up, down, left, or right) but cannot move diagonally.

Your task is to write a C++ program that checks if there exists a path for the rat to reach the food or not. You do not need to print the path itself, just determine if it's possible for the rat to reach the food.

You will need to make the following c:

CELL (class/struct):
The Cell class is defined to represent a cell in the maze. It contains the x and y coordinates of the cell and the dir variable to store the direction the search is currently moving in.

VISITED (2d array)
An array to keep track of visited cells.)

IsReachable (a function)
This function isReachable takes the maze as input and returns a boolean indicating whether a path from the start point to the target point exists. In this function you need to do the following things.

1. Initializes a stack and pushes the starting cell onto the stack.
2. The visited array is initialized with all cells marked as visited (set to true).
3. Inside the main loop (while stack not empty), it checks if the current cell is the target cell (fx and fy). If it is, the function returns true, indicating that a path is found. If not, it examines the neighboring cells in different directions (up, left, down, right) and pushes unvisited valid cells onto the stack, updating the visited array to mark them as visited.

Bonus:

You are given an array of item weights and an array of item values, along with the maximum weight capacity of a knapsack. The problem you are solving is known as the 0/1 Knapsack Problem. This problem involves selecting items to maximize the total value while staying within the weight constraint of the knapsack. Implement a C++ program to solve the 0/1 Knapsack Problem using a backtracking approach with a stack, without using vectors. Your program should find the maximum value that can be obtained without exceeding the weight capacity of the knapsack.

Explanation of the 0/1 Knapsack Problem:

The 0/1 Knapsack Problem is a classic optimization problem in computer science and mathematics. You are given a set of items, each with a weight and a value. The goal is to determine the combination of items to include in your knapsack (a container with a maximum weight capacity) in such a way that you maximize the total value while not exceeding the knapsack's weight capacity. Each item can only be selected once (0/1 choice), meaning you either include an item in the knapsack or leave it out.

Write a function **int knapsackBacktracking(int weights[], int values[], int numItems, int capacity)** that takes the following parameters:

1. weights: An integer array of item weights, where weights[i] (1 <= weights[i] <= 100) represents the weight of the i-th item.
2. values: An integer array of item values, corresponding to the weights in the weights array.
3. numItems: An integer representing the number of items (1 <= numItems <= 20).
4. capacity: An integer representing the maximum weight capacity of the knapsack (1 <= capacity <= 100).

The function should return the maximum value that can be obtained using a backtracking approach with a stack while adhering to the 0/1 Knapsack Problem constraints.

Constraints:

You must implement a backtracking algorithm with a stack.

```
#include <iostream>
using namespace std;

int knapsackBacktracking(int weights[], int values[], int numItems, int capacity);
```

```
int main() {
    int weights[] = {10, 20, 30};
    int values[] = {60, 100, 120};
    int numItems = 3;
    int capacity = 50;

    int max_value = knapsackBacktracking(weights, values, numItems, capacity);
    cout << "Maximum value: " << max_value << endl;

    return 0;
}
```

Output:

Maximum value: 220

Home Tasks (Non Graded)

1. Reverse a string using stacks
2. Implement a queue using stacks
3. Sort a stack using a temporary stack
4. Remove all duplicates of adjacent characters from a string using stacks.
5. Design a stack to support an additional operation that returns the minimum element from the stack in constant time. The stack should continue supporting all other operations like push, pop, top, size, empty, etc., with no degradation in these operations' performance.

6. Implement a queue using two stacks
7. Reverse a stack using recursion
8. Given a Queue consisting of first n natural numbers (in random order). The task is to check whether the given Queue elements can be arranged in increasing order in another Queue using a stack.